

Automation of Apache Spark Deployment with Ansible and Terraform on GCP

Dang Thai Son - 2440045

I/ Introduction

The system of this project relies on three main tools. Terraform sets up all virtual machines and networking components. Ansible configures those machines after they are created, installing Java, Spark, system utilities, and configuration files. Apache Spark acts as the distributed computation tool. The deployment requires SSH key-based authentication, a GCP service account for Terraform, and IAM permissions to manage compute and storage resources. A Cloud Storage (GCS) bucket is used as shared storage so the Spark cluster can read input data and write results in a common location accessible by all nodes.

The overall architecture of this project consists of a single Spark master node, two worker nodes, and one edge node. The edge machine is responsible for receiving the user's application and submitting it to the master. The workers are responsible for executing distributed tasks. All nodes communicate through a private subnet created by Terraform, which improves security and reflects the deployment of real distributed clusters.

II/ Deployment process

The deployment begins with GCP setup. A service account is created with proper permissions to manage compute instances and storage buckets. A JSON key for this account allows Terraform to authenticate automatically. APIs for Compute Engine and Cloud Storage are enabled. Local SSH keys are also generated so Ansible and SSH can access the cluster later.

Terraform is then used to create the entire infrastructure. A single virtual private cloud (VPC) is created, along with a subnet covering the 10.0.0.0/16 range. The master and worker nodes are created with private IP addresses only, while the edge node receives a public IP to serve as a gateway. All machines run Ubuntu 22.04 and use small to medium VM types (e2-small machine of 10GB for the edge and e2-medium machine of 20GB for the rest). After terraform apply, a script extracts the IP addresses to build an Ansible inventory that groups hosts into master, workers, and edge categories.

Because the worker and master nodes do not have public IPs, SSH traffic must pass through the edge node. A ProxyJump configuration is added to the SSH config on my local machine. With this setup, Ansible can reach all nodes transparently as if they were local hosts.

Ansible then installs all required software. Java, essential packages, and Spark are installed on every node. Configuration files such as `spark-env.sh` and `core-site.xml` are distributed automatically. The GCS connector is downloaded to the Spark installation directory on each node. This component is essential because Spark does not natively support GCS without it. After configuration, Ansible triggers the Spark master and worker startup scripts. The workers automatically register with the master, forming a functioning cluster. The Spark Web UI on port 8080 can be then accessed through an SSH tunnel from the operator's machine.

III/ Cluster validation

To validate the cluster, a WordCount program which is a map-reduce application that counts word occurrences in a text file is used.

A sample text file is uploaded to the GCS bucket, and the WordCount Java program is compiled into a JAR file. This JAR is then copied to the edge node. Using `spark-submit`, the application is submitted to the master node with input and output paths pointing to the GCS bucket. Because the GCS connector is installed on all Spark nodes, the cluster can directly read the input file and write results back to the bucket. Successful execution produces output files (e.g., `part-00000`, `part-00001`) which can be inspected using `gsutil`.

This validation confirms not only that Spark is installed correctly, but also that the cluster can handle distributed file systems across master and workers.

IV/ Performance testing

A larger experiment was carried out using a text file of roughly 1.1 GB. The goal was to evaluate the impact of cluster size on performance. Two cases were compared. In the first case, the full cluster with two workers was used. In the second case, execution was limited to a single worker by setting executor core limits during submission.

The results clearly show that using two workers improves performance. With two workers, the job completed in about 145 seconds, while the same task took about 199 seconds with only one worker. This represents more than 25% reduction in execution time due to parallelism.

V/ Conclusion

This project successfully automated the provisioning and configuration of a full Apache Spark cluster on Google Cloud Platform. Using Terraform and Ansible together ensures that the deployment is repeatable, and easy to maintain. Spark was configured to work flawlessly with Google Cloud Storage, enabling distributed applications to access a shared file system. The WordCount validation confirmed that the cluster operates correctly, and performance measurements showed clear speed improvements when running with additional worker nodes.